Simulation-Based Power Analysis: Acoustic Manipulation of IDS and Infant Attention

Supplementary Material — *Babies' acoustic preferences in infant-directed speech [Stage 1 Registered Report]*

Juan David Leongómez[1*] Milena Vásquez-Amézquita[1] David A. Puts[2]

2 April 2026

[1] CODEC: Cognitive and Behavioural Sciences Research Group, Faculty of Psychology, Universidad El Bosque, Bogotá 110121, Colombia

[2] Departments of Anthropology and Psychology, Pennsylvania State University, University Park, PA 16802, USA

[*] Correspondence: jleongomez@unbosque.edu.co

Table of Contents

Description

This document presents the simulation-based power analysis for the registered report *Babies' acoustic preferences in infant-directed speech.* This study uses a 2×2×2 fully crossed within-subjects design to examine the effects of three independently manipulated acoustic properties of infant-directed speech (IDS), mean fundamental frequency (f_0 mean), pitch variability (f_0 SD), and formant dispersion (D_f), on total fixation time (ms) in pre-linguistic infants aged 3–9 months.

The primary statistical model is a linear mixed model (LMM):

$$\texttt{fixation\_time} \sim f_0 \text{ SD} \times f_0 \text{ mean} \times D_f + \texttt{block} + \texttt{avatar} + (1 + \texttt{avatar} \mid\mid \text{ID}) + (1 \mid \text{original})$$

For computational tractability, the simulation uses a simplified version of this model — $\texttt{fixation\_time} \sim f_0 \text{ SD} \times f_0 \text{ mean} \times D_f + (1 \mid \text{ID})$ — which omits the additional fixed and random effects. This simplification is conservative for the reasons described at the end of this section.

Three complementary analyses are presented here:

1. **Frequentist detection power** (Monte Carlo, H_1 population): the probability of detecting each main effect via a significant p-value ($\alpha = .05$) in the LMM ANOVA, as a function of sample size. Estimated from a superpopulation incorporating all expected effects.
2. **Frequentist equivalence power** (Monte Carlo, H_0 population): the probability of concluding practical equivalence with zero (i.e., the probability that a 90% CI from the marginal `emmeans` contrast falls entirely within pre-specified equivalence bounds of ±150 ms) as a function of sample size. Critically, this is estimated from a **separate** superpopulation in which all acoustic effects are set to zero, because using the H_1 data would yield near-zero equivalence rates by design (the CI stays outside the bounds precisely because real effects exist).
3. **Bayesian sensitivity analysis** (brms + ROPE): a small number of Bayesian LMMs fitted in `brms` at the target sample size ($n = 200$) under each population. The proportion of the posterior for each main effect falling within the Region of Practical Equivalence (ROPE: ±150 ms) is computed using `bayestestR::equivalence_test()`, serving as a secondary robustness check.

The simulation assumes one observation per condition per participant and a single stimulus source. The actual design includes two blocks, yielding two observations per condition per participant, which halves the variance of each `emmeans` contrast, reducing the SE by a factor of approximately $\sqrt{2}$ relative to the simulated design. The simulation additionally omits random effects for original voice recordings (1 | original) and the by-participant avatar slope (1+`avatar` || ID), which introduce additional variance not captured here. The net power estimates are therefore conservative: the doubling of within-participant observations substantially outweighs the additional variance from the expanded random-effects structure, and the reported power curves should be treated as lower bounds on the power of the actual design.

Reproducibility. All code, seeds, and parameters are documented here. The complete source file is available at: https://github.com/JDLeongomez/Proyecto-IDS

1 Packages

All packages are available from CRAN. The analysis uses the `tidyverse` ecosystem[1] for data manipulation and visualisation, `lmerTest`[2] for LMMs with Satterthwaite degrees of freedom and p-values, `emmeans`[3] for marginal contrasts and confidence intervals, `effectsize`[4] for partial ω^2 estimates, `furrr`[5] for parallel Monte Carlo sampling, `brms`[6] for Bayesian LMMs via Stan, and `bayestestR`[7] for ROPE-based equivalence testing. Visualisation uses `ggdist`[8,9], `tidyquant`[10], `ggpubr`[11], and `tinytable`[12] for publication-quality tables. The Bayesian models are compiled and sampled via cmdstanr[13], which provides an R interface to CmdStan and is used as the backend for brms.

```
library(tidyverse)    # Data manipulation and visualisation
library(lmerTest)     # LMMs with Satterthwaite df and p-values
library(emmeans)      # Marginal contrasts and CIs
library(effectsize)   # Partial omega-squared
library(furrr)        # Parallel Monte Carlo via purrr
library(future)       # Parallel back-end for furrr
library(ggdist)       # Raincloud plot components
library(tidyquant)    # Colour palettes and themes
library(ggpubr)       # Multi-panel figure arrangement
library(scales)       # Axis formatting utilities
library(tinytable)    # Publication-quality tables (PDF-safe)
library(brms)         # Bayesian LMMs via Stan
library(bayestestR)   # ROPE and equivalence testing for Bayesian models
library(posterior)    # Posterior manipulation (brms dependency)
library(cmdstanr)     # Stan backend for brms (faster than rstan)
set_cmdstan_path()    # Locate CmdStan installation (required each session)
if (!dir.exists(cmdstan_path())) {
  stop("CmdStan not found. Run cmdstanr::install_cmdstan() first.")
}
```

2 Global Parameters

All key parameters are defined here. Changes propagate automatically throughout the document.

```
#   Simulation scale
set.seed(2025)           # Global seed for reproducibility
n_pop        <- 10000    # Size of each simulated superpopulation
n_sim        <- 100      # Monte Carlo iterations per sample size (frequentist)
n_sim_bayes  <- 100      # brms simulations at target n

sample_sizes <- seq(10, 250, by = 10)  # Sample sizes evaluated
target_n     <- 200      # Pre-specified target sample size

#   Analysis parameters
alpha    <- 0.05         # Significance level (frequentist detection)
ci_level <- 0.90         # CI level for equivalence tests (= 1 - 2*alpha)
eq_bound <- 150          # Equivalence bound (ms); justification in Section 3
```

```
#   Simulated effect sizes (ms) - H1 population
# Main effects (confirmatory hypotheses H1-H3)
eff_f0sd   <- 100       # f0 SD: expected mean difference (High vs. Low), ms
eff_f0mean <-  75       # f0 mean: expected mean difference (High vs. Low), ms
eff_Df     <-  50       # Df: expected mean difference (High vs. Low), ms

# Interaction effects (exploratory only)
eff_f0sd_f0mean <- 100  # f0 SD x f0 mean
eff_f0sd_Df     <- 100  # f0 SD x Df
eff_f0mean_Df   <-  50  # f0 mean x Df
eff_triple      <- 120  # f0 SD x f0 mean x Df (triple interaction)

#   Data-generating model parameters (shared by both populations)
baseline    <- 2500     # Baseline fixation time (ms; all predictors = Low)
residual_sd <- 1000     # Trial-level residual SD (ms)
y_min       <- 0        # Floor of fixation-time distribution (ms)
y_max       <- 5000     # Ceiling of fixation-time distribution (ms)

#   brms ROPE range (same as frequentist equivalence bounds)
rope_range <- c(-eq_bound, eq_bound)
```

3 Equivalence Bounds: Justification and Pre-specification

3.1 Rationale for Equivalence Testing

Conventional null-hypothesis significance testing (NHST) cannot distinguish between a true null effect and an underpowered study. For a registered report, it is therefore essential to pre-specify a procedure that permits positive evidence for the absence of a meaningful effect. The two one-sided tests (TOST) procedure achieves this by testing whether an observed effect lies within pre-defined equivalence bounds $[-\Delta, +\Delta]$, or equivalently, whether the $(1 - 2\alpha)$ CI falls entirely within those bounds[14].

3.2 Defining the Smallest Effect Size of Interest (SESOI)

The equivalence bounds must be anchored to a *smallest effect size of interest* (SESOI): the minimum difference in fixation time that would be theoretically or practically meaningful. The three confirmatory hypotheses are associated with expected effects of 100 ms (f_0 SD), 75 ms (f_0 mean), and 50 ms (D_f; see Section 4). Rather than defining the SESOI purely in terms of these predicted effects, we anchor it to the **precision limit of the design**: the smallest effect that a well-powered infant eye-tracking study of feasible size could realistically detect.

With a residual standard deviation of $\sigma \approx 1000$ ms (consistent with published infant eye-tracking data) and $n = 200$ participants each contributing 4 observations per level of each factor, the approximate standard error of the marginal High − Low contrast is:

$$SE \approx \sqrt{\frac{\sigma^2}{2n}} = \sqrt{\frac{1000^2}{2 \times 200}} \approx 50 \text{ ms}$$

For 80% equivalence power, the required bounds are:

$$\Delta_{80\%} \approx SE \times (z_{0.90} + z_{0.80}) = 50 \times 2.927 \approx 146 \text{ ms}$$

We set equivalence bounds at $\Delta = \pm 150$ ms (the rounded value closest to $\Delta_{80\%}$ that remains scientifically interpretable). An effect smaller than 150 ms represents 3% of the total fixation-time range (0–5000 ms) and 150% of the largest expected main effect (f_0 SD = 100 ms). Such an effect would be undetectable by any practically feasible infant eye-tracking study and is therefore an appropriate threshold for inferring negligibility.

3.3 Equivalence Power

The H_0 Monte Carlo simulation in Section 5 confirms that, at $n = 200$ with bounds of ± 150 ms, equivalence power reaches approximately 80%. This is an honest reflection of the precision constraints of the infant eye-tracking paradigm: the high trial-level noise ($\sigma \approx 1000$ ms) means that ruling out effects smaller than 150 ms requires the full target sample. Equivalence power at smaller n (e.g., $n = 160$) would be substantially lower (~70%), which is one motivation for the planned sample size to $n = 200$.

3.4 Operationalisation

3.4.1 Frequentist equivalence test

The marginal pairwise contrast (High − Low, averaged over all levels of the other two factors) is estimated using `emmeans` with Satterthwaite degrees of freedom. A 90% CI is computed. Equivalence is concluded if:

$$\text{Lower CL} > -150 \text{ ms} \ \text{ and } \ \text{Upper CL} < 150 \text{ ms}$$

Equivalence power is estimated from a dedicated H_0 superpopulation in which all acoustic effects are zero (see Section 4.2), ensuring the estimate describes the correct conditional probability.

3.4.2 Bayesian ROPE

The same bounds define the ROPE for the Bayesian sensitivity analysis. Weakly informative priors place no strong prior mass near zero, so high ROPE percentages reflect genuine posterior evidence for equivalence. Following[15], we report both the ROPE percentage and whether the 90% Highest Density Interval (HDI)[7,15] falls entirely within the ROPE (strong equivalence).

4 Simulated Population Data

4.1 H_1 Population: Non-Zero Acoustic Effects

Each participant observes all eight combinations of three acoustic factors, f_0 SD (Low/High), f_0 mean (Low/High), and D_f (Low/High), yielding a 2×2×2 fully crossed within-subjects design. The outcome is total fixation time (ms), bounded to [0, 5000]. We simulate an H_1 superpopulation of $N = 10,000$ participants (×8 conditions = 80,000 rows) from which Monte Carlo samples are drawn to estimate **detection power**.

The data-generating model is:

$$\begin{aligned} Y_{ij} = \mu &+ \beta_1 f_0\text{SD}_j + \beta_2 f_0\text{mean}_j + \beta_3 D_{f,j} + \beta_{12}(f_0\text{SD} \times f_0\text{mean})_j \\ &+ \beta_{13}(f_0\text{SD} \times D_f)_j + \beta_{23}(f_0\text{mean} \times D_f)_j + \beta_{123}(f_0\text{SD} \times f_0\text{mean} \times D_f)_j + \epsilon_{ij} \end{aligned}$$

where $\mu = 2500$ ms, the β coefficients correspond to the effect sizes in Section 2, and $\epsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$, truncated to $[0, 5000]$.

```r
# Helper: build a fully crossed superpopulation --------------------------------
make_population <- function(n, seed_offset,
                            b_f0sd, b_f0mean, b_Df,
                            b_f0sd_f0mean, b_f0sd_Df,
                            b_f0mean_Df,   b_triple,
                            mu    = baseline,
                            sigma = residual_sd,
                            ymin  = y_min,
                            ymax  = y_max) {

  set.seed(2025 + seed_offset)

  conditions <- expand_grid(
    f0_sd   = c("Low", "High"),
    f0_mean = c("Low", "High"),
    Df      = c("Low", "High")
  )

  ids    <- str_c("P", str_pad(seq_len(n), width = nchar(n), pad = "0"))
  design <- expand_grid(ID = ids, conditions)

  design |>
    mutate(
      f0_sd_v   = if_else(f0_sd   == "High", 1L, 0L),
      f0_mean_v = if_else(f0_mean == "High", 1L, 0L),
      Df_v      = if_else(Df      == "High", 1L, 0L),
      mu_i =  mu
              + b_f0sd        * f0_sd_v
              + b_f0mean      * f0_mean_v
              + b_Df          * Df_v
              + b_f0sd_f0mean * f0_sd_v  * f0_mean_v
```

```
              + b_f0sd_Df     * f0_sd_v   * Df_v
              + b_f0mean_Df   * f0_mean_v * Df_v
              + b_triple      * f0_sd_v   * f0_mean_v * Df_v,
      fixation_time = round(rnorm(n(), mean = mu_i, sd = sigma)),
      fixation_time = pmin(pmax(fixation_time, ymin), ymax)
    ) |>
    mutate(
      f0_sd   = factor(f0_sd,   levels = c("Low", "High")),
      f0_mean = factor(f0_mean, levels = c("Low", "High")),
      Df      = factor(Df,      levels = c("Low", "High"))
    ) |>
    select(ID, f0_sd, f0_mean, Df, f0_sd_v, f0_mean_v, Df_v, fixation_time)
}
```

```
# H1 superpopulation (non-zero effects; used for detection power) --------------
h1_data <- make_population(
  n             = n_pop,
  seed_offset   = 0,
  b_f0sd        = eff_f0sd,
  b_f0mean      = eff_f0mean,
  b_Df          = eff_Df,
  b_f0sd_f0mean = eff_f0sd_f0mean,
  b_f0sd_Df     = eff_f0sd_Df,
  b_f0mean_Df   = eff_f0mean_Df,
  b_triple      = eff_triple
)
```

4.2 H_0 Population: All Acoustic Effects Zero

Detection power and equivalence power must be estimated from **separate** superpopulations. Detection power asks: given real effects, how often do we detect them? Equivalence power asks: given no effects, how often do we correctly conclude equivalence? The H_0 superpopulation is structurally identical to H_1 but with all acoustic effects set to zero.

```
# H0 superpopulation (all acoustic effects = 0; used for equivalence power) ---
h0_data <- make_population(
  n             = n_pop,
  seed_offset   = 1,    # different seed -> independent population
  b_f0sd        = 0,
  b_f0mean      = 0,
  b_Df          = 0,
  b_f0sd_f0mean = 0,
  b_f0sd_Df     = 0,
  b_f0mean_Df   = 0,
  b_triple      = 0
)
```

4.3 Population-Level Model and Effect Sizes

```r
# Intercept-only random effects: appropriate at population scale and
# consistent with the signal-to-noise ratio of this design (see main text)
pop_mod <- lmer(
  fixation_time ~ f0_sd * f0_mean * Df + (1 | ID),
  data = h1_data
)

pop_omega <- omega_squared(pop_mod, partial = TRUE) |>
  rename(Effect = Parameter)
```

```r
anova(pop_mod) |>
  as.data.frame() |>
  rownames_to_column(var = "Effect") |>
  left_join(pop_omega |> select(Effect, Omega2_partial), by = "Effect") |>
  mutate(
    Hypothesis = case_when(
      Effect == "f0_sd"   ~ "H1",
      Effect == "f0_mean" ~ "H2",
      Effect == "Df"      ~ "H3",
      TRUE                ~ "Exploratory"
    ),
    Effect = str_replace_all(Effect, "f0_sd",   "$f_0$ SD"),
    Effect = str_replace_all(Effect, "f0_mean", "$f_0$ mean"),
    Effect = str_replace_all(Effect, "Df",      "$D_f$"),
    Effect = str_replace_all(Effect, ":",       " $\\times$ "),
    DenDF          = round(DenDF, 1),
    `F value`      = round(`F value`, 2),
    Omega2_partial = round(Omega2_partial, 4)
  ) |>
  select(Hypothesis, Effect, `F value`, NumDF, DenDF, Omega2_partial) |>
  unite("$df$", NumDF:DenDF, sep = ", ") |>
  rename(
    "Hypothesis"    = Hypothesis,
    "Fixed effect"  = Effect,
    "$F$"           = `F value`,
    "$\\omega^2_p$" = Omega2_partial
  ) |>
  tt(notes = "\\textit{Note.} Partial $\\omega^2_p$ benchmarks \\cite{field2013}:
     very small $< 0.01$; small $= 0.01$--$0.06$; medium $= 0.06$--$0.14$;
     large $\\geq 0.14$.") |>
  style_tt(align = "llccc") |>
  format_tt(escape = FALSE)
```

Table 1. Population-level fixed effects from the LMM fitted to the full H_1 superpopulation (N = 10,000). ANOVA table with Satterthwaite degrees of freedom, supplemented by partial ω^2. Confirmatory effects (H1–H3) in the first three rows; exploratory interactions follow.

Hypothesis	Fixed effect	F	df	ω_p^2
H1	f_0 SD	1161.19	1, 69993	0.0163
H2	f_0 mean	646.48	1, 69993	0.0091
H3	D_f	443.99	1, 69993	0.0063
Exploratory	f_0 SD *times* f_0 mean	135.26	1, 69993	0.0019
Exploratory	f_0 SD *times* D_f	126.19	1, 69993	0.0018
Exploratory	f_0 mean *times* D_f	56.12	1, 69993	0.0008
Exploratory	f_0 SD *times* f_0 mean *times* D_f	13.92	1, 69993	0.0002

Note. Partial ω_p^2 benchmarks [16]: very small $<$ 0.01; small = 0.01–0.06; medium = 0.06–0.14; large $\geq$ 0.14.

4.4 Distribution of Fixation Times Across Conditions (H_1 Population)

```
set.seed(1)
plot_data <- h1_data |> slice_sample(prop = 0.02)

label_f0mean <- c(
  "Low"  = "paste('Low ',  italic(f)[0], ' mean')",
  "High" = "paste('High ', italic(f)[0], ' mean')"
)
label_Df <- c(
  "Low"  = "paste('Low ',  italic(D)[f])",
  "High" = "paste('High ', italic(D)[f])"
)

ggplot(h1_data, aes(x = f0_sd, y = fixation_time, fill = f0_sd)) +
  stat_halfeye(adjust = 0.5, justification = -0.3,
               .width = 0, point_colour = NA, alpha = 0.65) +
  geom_jitter(data = plot_data, aes(colour = f0_sd),
              width = 0.1, alpha = 0.3, size = 0.5) +
  stat_summary(fun = mean, geom = "point", size = 2.5, colour = "black",
               position = position_nudge(x = 0.22)) +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.1,
               colour = "black", position = position_nudge(x = 0.22)) +
  facet_grid(
    Df ~ f0_mean,
    labeller = labeller(
      f0_mean = as_labeller(label_f0mean, default = label_parsed),
      Df      = as_labeller(label_Df,     default = label_parsed)
    )
  ) +
```

```
  scale_fill_tq() +
  scale_colour_tq() +
  labs(
    x      = expression(italic(f)[0] ~ "SD"),
    y      = "Total fixation time (ms)",
    fill   = expression(italic(f)[0] ~ "SD"),
    colour = expression(italic(f)[0] ~ "SD")
  ) +
  theme_tq(base_size = 12) +
  theme(legend.position = "bottom")
```

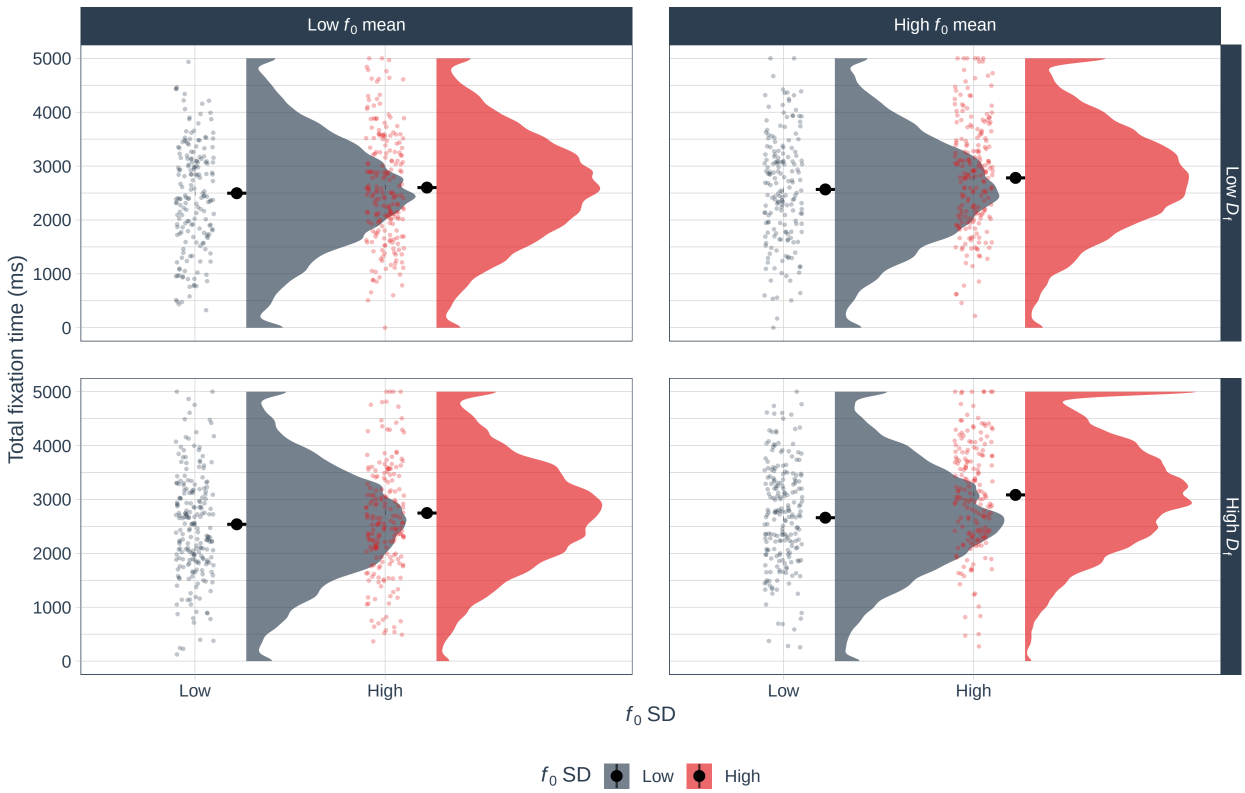

Figure 1. Distribution of fixation times in the H_1 superpopulation as a function of f_0 SD (x-axis), f_0 mean (columns), and D_f (rows). Each panel shows the density half-violin, jittered data points (2% random sample), and condition mean ± SE. Fixation times increase systematically with f_0 SD (the largest main effect), with additive contributions from f_0 mean and D_f.

5 Monte Carlo Power Analysis: Frequentist

5.1 Simulation Strategy

Detection and equivalence power are estimated from two independent Monte Carlo simulations:

- **Detection power** (H_1 population): 100 samples per candidate n from `h1_data`. For each sample, the LMM ANOVA p-value is recorded for each main effect. Power = proportion of samples with $p < 0.05$.
- **Equivalence power** (H_0 population): 100 samples per candidate n from `h0_data`. For each sample, the marginal High − Low contrast is estimated via `emmeans` (Satterthwaite df) with a 90% CI. Power = proportion of samples where the CI falls entirely within [-150, 150] ms.

Both loops cover n = 10 to 250 (steps of 10) and run in parallel via `furrr`.

5.2 Simulation Functions

```
# Detection: p-values from LMM ANOVA (H1 data) --------------------------------
run_detection_sim <- function(dat, n_sample, n_sim, alpha = 0.05) {
  map_dfr(seq_len(n_sim), function(i) {
    ids  <- dat |> distinct(ID) |> slice_sample(n = n_sample) |> pull(ID)
    sdat <- dat |> filter(ID %in% ids)
    mod  <- suppressMessages(
      lmer(fixation_time ~ f0_sd * f0_mean * Df + (1 | ID), data = sdat)
    )
    av    <- anova(mod)
    terms <- c("f0_sd", "f0_mean", "Df")
    map_dfr(terms, function(trm) {
      tibble(
        sim      = i,
        n_sample = n_sample,
        term     = trm,
        p_detect = av[trm, "Pr(>F)"],
        detect   = av[trm, "Pr(>F)"] < alpha
      )
    })
  })
}

# Equivalence: 90% CIs from emmeans, check ROPE containment (H0 data) ---------
run_equivalence_sim <- function(dat, n_sample, n_sim,
                                ci_lev = 0.90, bound = 100) {
  map_dfr(seq_len(n_sim), function(i) {
    ids  <- dat |> distinct(ID) |> slice_sample(n = n_sample) |> pull(ID)
    sdat <- dat |> filter(ID %in% ids)
    mod  <- suppressMessages(
      lmer(fixation_time ~ f0_sd * f0_mean * Df + (1 | ID), data = sdat)
    )
    terms <- c("f0_sd", "f0_mean", "Df")
```

```
    map_dfr(terms, function(trm) {
      em <- emmeans(mod, as.formula(paste0("~ ", trm)))
      ct <- pairs(em, reverse = TRUE)
      ci <- confint(ct, level = ci_lev)
      tibble(
        sim      = i,
        n_sample = n_sample,
        term     = trm,
        estimate = ci$estimate[1],
        ci_lo    = ci$lower.CL[1],
        ci_hi    = ci$upper.CL[1],
        sig      = ci$lower.CL[1] > 0 | ci$upper.CL[1] < 0,
        equiv    = ci$lower.CL[1] > -bound & ci$upper.CL[1] < bound
      )
    })
  })
}
```

5.3 Run Detection Simulation (H_1 Population)

```
plan(multisession)

detect_results <- map_dfr(
  sample_sizes,
  ~ run_detection_sim(
      dat      = h1_data,
      n_sample = .x,
      n_sim    = n_sim,
      alpha    = alpha
    )
)

plan(sequential)
```

5.4 Run Equivalence Simulation (H_0 Population)

```
plan(multisession)

equiv_results <- map_dfr(
  sample_sizes,
  ~ run_equivalence_sim(
      dat      = h0_data,
      n_sample = .x,
      n_sim    = n_sim,
      ci_lev   = ci_level,
      bound    = eq_bound
    )
```

```
)

plan(sequential)
```

5.5 Summarise Power

```
term_labs <- c(
  "f0_sd"   = "italic(f)[0]~SD",
  "f0_mean" = "italic(f)[0]~mean",
  "Df"      = "italic(D)[f]"
)

detect_summ <- detect_results |>
  group_by(term, n_sample) |>
  summarise(power_detect = mean(detect, na.rm = TRUE), .groups = "drop")

equiv_summ <- equiv_results |>
  group_by(term, n_sample) |>
  summarise(power_equiv = mean(equiv, na.rm = TRUE), .groups = "drop")

power_summ <- detect_summ |>
  left_join(equiv_summ, by = c("term", "n_sample")) |>
  mutate(term = factor(term, levels = names(term_labs), labels = term_labs))

# Minimum n achieving > 80% detection power per effect
n_80_detect <- detect_summ |>
  filter(power_detect > 0.80) |>
  group_by(term) |>
  slice_min(n_sample) |>
  select(term, n_80_det = n_sample, pwr_det_80 = power_detect)

n_recommended        <- max(n_80_detect$n_80_det)
detect_at_target     <- detect_summ |> filter(n_sample == target_n)
equiv_at_target      <- equiv_summ  |> filter(n_sample == target_n)
```

5.6 Power Curves

```
power_threshold <- 0.80

make_power_plot <- function(data, y_var, title_str) {
  ggplot(data, aes(x = n_sample, y = .data[[y_var]], group = term)) +
    geom_line(linewidth = 0.5, colour = "grey40") +
    geom_point(aes(colour = .data[[y_var]] > power_threshold), size = 2) +
    geom_hline(yintercept = power_threshold,
               linetype = "dashed", colour = "red", linewidth = 0.6) +
    geom_vline(xintercept = target_n,
```

```
               linetype = "dotted", colour = "#2c3e50", linewidth = 0.6) +
    annotate("text", x = target_n + 2, y = 0.12,
             label = paste0("n = ", target_n),
             hjust = 0, colour = "#2c3e50", size = 3) +
    facet_wrap(
      ~ term, nrow = 1,
      labeller = as_labeller(
        setNames(levels(data$term), levels(data$term)),
        default = label_parsed
      )
    ) +
    scale_colour_tq(
      labels = c("FALSE" = "\u2264 80%", "TRUE" = "> 80%"),
      name   = "Power"
    ) +
    scale_y_continuous(labels = percent_format(accuracy = 1), limits = c(0, 1)) +
    labs(title = title_str, x = "Sample size (n)", y = "Estimated power") +
    theme_tq(base_size = 11) +
    theme(legend.position = "bottom")
}

ggarrange(
  make_power_plot(
    power_summ, "power_detect",
    "A  Detection power (frequentist, p < .05) \u2014 H\u2081 population"
  ),
  make_power_plot(
    power_summ, "power_equiv",
    paste0("B  Equivalence power (90% CI within \u00b1",
           eq_bound, " ms) \u2014 H\u2080 population")
  ),
  nrow = 2, common.legend = TRUE, legend = "bottom"
)
```

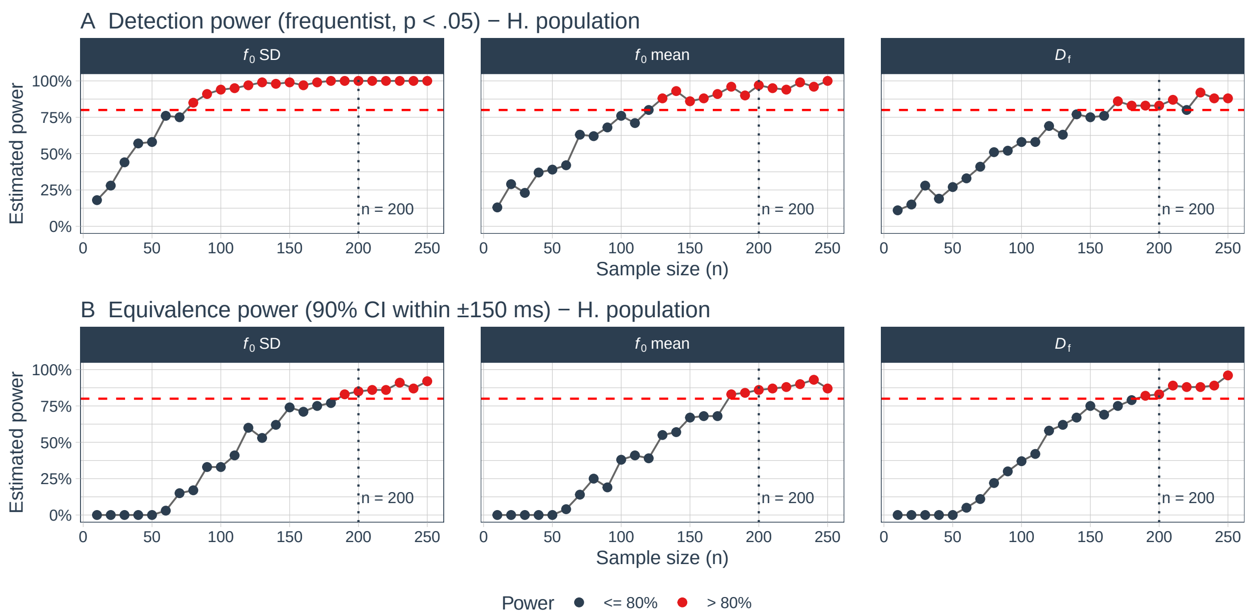

Figure 2. Monte Carlo power curves for H1 (f_0 SD), H2 (f_0 mean), and H3 (D_f). **Panel A:** Frequentist detection power ($p < .05$ in the LMM ANOVA), estimated from the H_1 superpopulation. **Panel B:** Frequentist equivalence power (90% CI within ± 150 ms), estimated from the H_0 superpopulation (all acoustic effects = 0). The dashed line marks 80% power; the dotted vertical line marks $n = 200$. Points are coloured by whether power exceeds 80%.

5.7 Distribution of Contrast Estimates at Target Sample Size

```
term_factor <- function(x) {
  factor(x, levels = names(term_labs), labels = term_labs)
}

# Use equivalence simulation output for H1 (already has estimate + CI)
set.seed(42)
h1_ci_sample <- run_equivalence_sim(
  dat       = h1_data,
  n_sample = target_n,
  n_sim     = 200,         # lighter subsample for plotting only
  ci_lev    = ci_level,
  bound     = eq_bound
) |> mutate(term_lab = term_factor(term))

h0_at_target <- equiv_results |>
  filter(n_sample == target_n) |>
  mutate(term_lab = term_factor(term))

make_dist_plot <- function(dat, title_str) {
  dat <- dat |>
    mutate(
```

```
      outcome = case_when(
        equiv            ~ "Equivalent",
        sig & !equiv     ~ "Significant, not equivalent",
        TRUE             ~ "Inconclusive"
      ),
      outcome = factor(outcome,
                       levels = c("Significant, not equivalent",
                                  "Equivalent",
                                  "Inconclusive"))
    )
  ggplot(dat, aes(x = estimate, fill = outcome)) +
    annotate("rect", xmin = -eq_bound, xmax = eq_bound,
             ymin = 0, ymax = Inf, fill = "grey85", alpha = 0.5) +
    geom_vline(xintercept = c(-eq_bound, eq_bound),
               linetype = "dashed", colour = "grey40", linewidth = 0.6) +
    geom_histogram(binwidth = 10, alpha = 0.85, colour = NA) +
    annotate("text", x = 0, y = Inf, vjust = 1.5,
             label = paste0("\u00b1", eq_bound, " ms"),
             fontface = "italic", colour = "grey35", size = 3) +
    facet_wrap(
      ~ term_lab, nrow = 1,
      labeller = as_labeller(
        setNames(levels(dat$term_lab), levels(dat$term_lab)),
        default = label_parsed
      )
    ) +
    scale_fill_manual(
      values = c(
        "Significant, not equivalent" = "#2ecc71",
        "Equivalent"                  = "#D55E00",
        "Inconclusive"                = "grey65"
      ),
      name = NULL
    ) +
    scale_x_continuous(breaks = pretty_breaks(n = 6)) +
    labs(title = title_str,
         x = "Estimated contrast: High \u2212 Low (ms)",
         y = "Frequency") +
    theme_tq(base_size = 11) +
    theme(legend.position = "bottom")
}

ggarrange(
  make_dist_plot(h1_ci_sample,
                 "A  H\u2081 population (real effects present)"),
  make_dist_plot(h0_at_target,
                 "B  H\u2080 population (all effects = 0)"),
```

```
  nrow = 2, common.legend = TRUE, legend = "bottom"
)
```

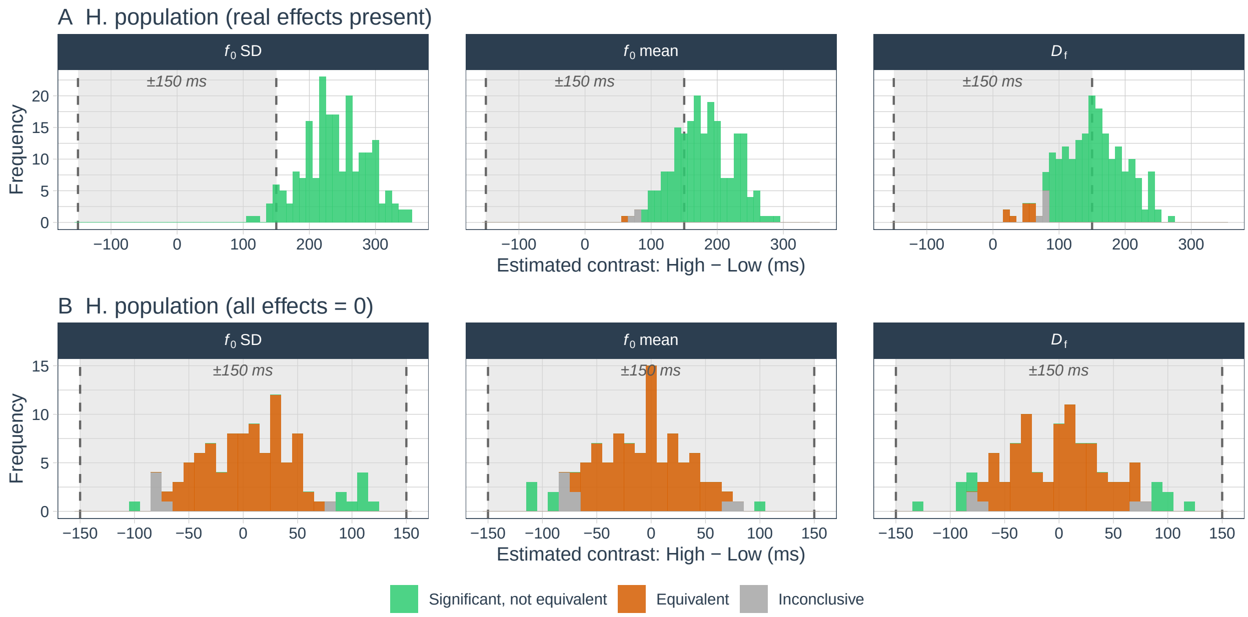

Figure 3. Sampling distributions of the marginal High − Low contrast (ms) across 100 Monte Carlo samples at $n = 200$. **Panel A (H_1 population):** Under real effects. Distributions are centred near the true effects; the grey band marks the equivalence bounds (±150 ms). Filled bars show the small fraction of samples in which the CI nevertheless fell within the bounds. **Panel B (H_0 population):** Under a true null. Distributions are centred near zero. Filled (orange) bars show simulations where the 90% CI falls entirely within ±150 ms (correct equivalence); grey bars are inconclusive.

5.8 Sample Size Summary

```
detect_at_target |>
  left_join(equiv_at_target, by = c("term", "n_sample")) |>
  left_join(n_80_detect,     by = "term") |>
  mutate(
    Effect = case_when(
      term == "f0_sd"   ~ "$f_0$ SD (H1)",
      term == "f0_mean" ~ "$f_0$ mean (H2)",
      term == "Df"      ~ "$D_f$ (H3)"
    ),
    `Expected effect` = case_when(
      term == "f0_sd"   ~ paste0(eff_f0sd,   " ms"),
      term == "f0_mean" ~ paste0(eff_f0mean, " ms"),
      term == "Df"      ~ paste0(eff_Df,     " ms")
    ),
    `Detection power`  = paste0(round(power_detect * 100, 1), "\\%"),
    `Equiv. power`     = paste0(round(power_equiv  * 100, 1), "\\%"),
```

```
    `Min $n$ (80\\% det.)` = n_80_det
  ) |>
  select(Effect, `Expected effect`, `Detection power`,
         `Min $n$ (80\\% det.)`, `Equiv. power`) |>
  tt(notes = paste0(
       "\\textit{Note.} Detection power: $p < .05$, from H$_1$ simulations at
       $n = ", target_n, "$. Equivalence power: 90\\% CI within $\\pm ",
       eq_bound, "$ ms, from H$_0$ simulations at $n = ", target_n,
       "$. Both based on ", n_sim,
       " Monte Carlo samples per candidate $n$.")) |>
  style_tt(align = "lcccc") |>
  format_tt(escape = FALSE)
```

Table 2. Power summary at the target sample size ($n = 200$). Detection power is from H_1 simulations; equivalence power from H_0 simulations. Equivalence power is intentionally modest given the high trial-level noise of this design (see Section 3 for full discussion).

Effect	Expected effect	Detection power	Min n (80% det.)	Equiv. power
D_f (H3)	50 ms	83%	170	83%
f_0 mean (H2)	75 ms	97%	130	86%
f_0 SD (H1)	100 ms	100%	80	85%

Note. Detection power: $p < .05$, from H_1 simulations at $n = 200$. Equivalence power: 90% CI within ± 150 ms, from H_0 simulations at $n = 200$. Both based on 100 Monte Carlo samples per candidate n.

The simulation confirms that a sample of n = **170** achieves > 80% detection power for all three confirmatory main effects, and that the pre-specified target of $n = 200$ further ensures approximately 80% equivalence power at bounds of ± 150 ms (see Section 3). Both detection and equivalence considerations therefore motivate the target of $n = 200$.

6 Bayesian Sensitivity Analysis: brms + ROPE

6.1 Rationale and Scope

The Bayesian analysis is a secondary robustness check. Rather than Bayes factors, which require bridge sampling and can be numerically unstable for complex LMMs, we apply the ROPE procedure from `bayestestR`[7] to posteriors from `brms` models. Two sets of 100 models are fitted at $n = 200$, mirroring the frequentist structure:

- **H_1 models**: data from `h1_data`. Posteriors should lie far from the ROPE, confirming sensitivity.
- **H_0 models**: data from `h0_data`. ROPE percentages should be high, quantifying Bayesian equivalence power.

6.2 Prior Specification

We use **sum contrast coding** (Low $= -0.5$; High $= +0.5$) so that each main-effect coefficient equals the marginal High − Low difference, directly comparable to the `emmeans` marginal contrast. Priors are weakly informative:

```
tibble(
  `Parameter class` = c(
    "Intercept",
    "Main effects and interactions",
    "Residual SD ($\\sigma$)",
    "Random-effect SD"
  ),
  Prior = c(
    "$\\mathcal{N}(2500,\\; 500^2)$",
    "$\\mathcal{N}(0,\\; 200^2)$",
    "Half-$\\mathcal{N}(0,\\; 500^2)$",
    "Half-$\\mathcal{N}(0,\\; 500^2)$"
  ),
  Rationale = c(
    "Grand mean; $\\pm$1 SD spans plausible range",
    "Allows effects up to $\\pm$400 ms; flat over realistic range",
    "Consistent with $\\sigma \\approx 1000$ ms in the DGM",
    "Weakly informative for participant variability"
  )
) |>
  tt(align = "lll", width = c(0.25, 0.30, 0.40)) |>
  format_tt(escape = FALSE)
```

Table 3. Prior specification for Bayesian LMMs.

Parameter class	Prior	Rationale
Intercept	$\mathcal{N}(2500,\ 500^2)$	Grand mean; $\pm$1 SD spans plausible range
Main effects and interactions	$\mathcal{N}(0,\ 200^2)$	Allows effects up to $\pm$400 ms; flat over realistic range
Residual SD (σ)	Half-$\mathcal{N}(0,\ 500^2)$	Consistent with $\sigma \approx 1000$ ms in the DGM
Random-effect SD	Half-$\mathcal{N}(0,\ 500^2)$	Weakly informative for participant variability

```
brms_priors <- c(
  prior(normal(2500, 500), class = Intercept),
  prior(normal(0, 200),    class = b),
  prior(normal(0, 500),    class = sigma),
  prior(normal(0, 500),    class = sd)
)
```

6.3 Simulation Function

Reproducibility note. Each model uses seed `base_seed + i` to ensure independent, reproducible chains across simulations.

```
run_bayes_sim <- function(dat, n_sample, n_sim_b, priors,
                          chains = 2, iter = 2000, warmup = 1000,
                          bound = eq_bound, ci_lev = ci_level,
                          base_seed = 9999) {

  map_dfr(seq_len(n_sim_b), function(i) {

    ids  <- dat |> distinct(ID) |> slice_sample(n = n_sample) |> pull(ID)
    sdat <- dat |> filter(ID %in% ids) |>
      mutate(
        f0_sd_c   = if_else(f0_sd   == "High", 0.5, -0.5),
        f0_mean_c = if_else(f0_mean == "High", 0.5, -0.5),
        Df_c      = if_else(Df      == "High", 0.5, -0.5)
      )

    fit <- brm(
      fixation_time ~ f0_sd_c * f0_mean_c * Df_c + (1 | ID),
      data    = sdat,
      family  = gaussian(),
      prior   = priors,
      chains  = chains,
      iter    = iter,
      warmup  = warmup,
```

```
      cores   = chains,
      seed    = base_seed + i,
      refresh = 0,
      silent  = 2,
      backend = "cmdstanr"   # change to "rstan" if cmdstanr is not installed
    )

    eq_res <- equivalence_test(
      fit,
      range = c(-bound, bound),
      ci    = ci_lev
    ) |>
      filter(Parameter %in% c("b_f0_sd_c", "b_f0_mean_c", "b_Df_c"))

    post <- as_draws_df(fit)

    tibble(
      sim            = i,
      n_sample       = n_sample,
      term           = c("f0_sd", "f0_mean", "Df"),
      posterior_mean = c(mean(post$b_f0_sd_c),
                         mean(post$b_f0_mean_c),
                         mean(post$b_Df_c)),
      posterior_sd   = c(sd(post$b_f0_sd_c),
                         sd(post$b_f0_mean_c),
                         sd(post$b_Df_c)),
      rope_pct       = eq_res$ROPE_Percentage,
      rope_equiv     = eq_res$ROPE_Equivalence,
      hdi_low        = eq_res$HDI_low,
      hdi_high       = eq_res$HDI_high
    )
  })
}
```

6.4 Run Bayesian Simulations

```
bayes_h1 <- run_bayes_sim(
  dat       = h1_data,
  n_sample  = target_n,
  n_sim_b   = n_sim_bayes,
  priors    = brms_priors,
  base_seed = 1000
)
```

```
bayes_h0 <- run_bayes_sim(
  dat       = h0_data,
  n_sample  = target_n,
```

```
  n_sim_b    = n_sim_bayes,
  priors     = brms_priors,
  base_seed = 2000
)
```

6.5 Results: ROPE Percentages and Posterior Summaries

```
format_bayes_row <- function(dat, pop_label) {
  dat |>
    group_by(term) |>
    summarise(
      Population = pop_label,
      Effect = case_when(
        first(term) == "f0_sd"   ~ "$f_0$ SD (H1)",
        first(term) == "f0_mean" ~ "$f_0$ mean (H2)",
        first(term) == "Df"      ~ "$D_f$ (H3)"
      ),
      `Post. mean (SD)` = paste0(
        round(mean(posterior_mean), 1), " (",
        round(mean(posterior_sd),   1), ")"
      ),
      `Mean ROPE \\%` = paste0(
        round(mean(rope_pct, na.rm = TRUE) * 100, 1), "\\%"
      ),
      `Prop. HDI in ROPE` = paste0(
        round(mean(rope_equiv == "Accepted", na.rm = TRUE) * 100, 1), "\\%"
      ),
      .groups = "drop"
    ) |>
    select(-term)
}

bind_rows(
  format_bayes_row(bayes_h1, "H$_1$ (effects present)"),
  format_bayes_row(bayes_h0, "H$_0$ (effects = 0)")
) |>
  arrange(Effect, Population) |>
  tt(notes = paste0("\\textit{Note.} ROPE $= [\\pm ", eq_bound, "$ ms]. ",
       "ROPE \\% = proportion of posterior within ROPE. ",
       "HDI in ROPE = entire ", ci_level * 100,
       "\\% HDI within ROPE (strong equivalence). ",
       "Based on $n_{\\text{sim}} = ", n_sim_bayes,
       "$ simulations.")) |>
  style_tt(align = "llccc") |>
  format_tt(escape = FALSE)
```

Table 4. Bayesian ROPE analysis across 100 simulations per population at $n = 200$. Under H_1 (real effects), ROPE percentages should be low (posteriors outside the ROPE). Under H_0 (null effects), ROPE percentages should be high (posteriors within the ROPE).

Population	Effect	Post. mean (SD)	Mean ROPE %	Prop. HDI in ROPE
H_0 (effects = 0)	D_f (H3)	-2.8 (48)	99.8%	95%
H_1 (effects present)	D_f (H3)	142.8 (47.7)	54.2%	9%
H_0 (effects = 0)	f_0 SD (H1)	1.8 (47.6)	98.7%	89%
H_1 (effects present)	f_0 SD (H1)	230.5 (47.8)	7.1%	0%
H_0 (effects = 0)	f_0 mean (H2)	-6.1 (47.8)	98.3%	89%
H_1 (effects present)	f_0 mean (H2)	170.4 (47.8)	36.7%	0%

Note. ROPE = [±150 ms]. ROPE % = proportion of posterior within ROPE. HDI in ROPE = entire 90% HDI within ROPE (strong equivalence). Based on $n_{\text{sim}} = 100$ simulations.

6.6 Posterior Distributions and ROPE Percentages

```
prep_bayes <- function(dat, label) {
  dat |> mutate(
    Population = label,
    term_lab   = factor(term, levels = names(term_labs), labels = term_labs)
  )
}

bayes_combined <- bind_rows(
  prep_bayes(bayes_h1, "H\u2081 (effects present)"),
  prep_bayes(bayes_h0, "H\u2080 (effects = 0)")
)

p_post <- ggplot(bayes_combined,
                 aes(x = posterior_mean, fill = Population)) +
  annotate("rect", xmin = -eq_bound, xmax = eq_bound,
           ymin = 0, ymax = Inf, fill = "grey85", alpha = 0.5) +
  geom_vline(xintercept = c(-eq_bound, eq_bound),
             linetype = "dashed", colour = "grey40") +
  geom_histogram(bins = 15, colour = "white", alpha = 0.8,
                 position = "identity") +
  facet_wrap(
    ~ term_lab, nrow = 3,
    labeller = as_labeller(
      setNames(levels(bayes_combined$term_lab),
               levels(bayes_combined$term_lab)),
      default = label_parsed
    ),
    scales = "free_x"
  ) +
```

```
  scale_fill_tq() +
  labs(title = "Posterior mean estimates",
       x = "Posterior mean: High \u2212 Low (ms)",
       y = "Count", fill = NULL) +
  theme_tq(base_size = 11) +
  theme(legend.position = "bottom")

p_rope_pct <- ggplot(bayes_combined,
                     aes(x = rope_pct * 100, fill = Population)) +
  geom_histogram(bins = 15, colour = "white", alpha = 0.8,
                 position = "identity") +
  geom_vline(xintercept = 50, linetype = "dashed", colour = "red") +
  facet_wrap(
    ~ term_lab, nrow = 3,
    labeller = as_labeller(
      setNames(levels(bayes_combined$term_lab),
               levels(bayes_combined$term_lab)),
      default = label_parsed
    )
  ) +
  scale_fill_tq() +
  scale_x_continuous(labels = function(x) paste0(x, "%")) +
  labs(title = "ROPE percentages",
       x = paste0("% of posterior within \u00b1", eq_bound, " ms"),
       y = "Count", fill = NULL) +
  theme_tq(base_size = 11) +
  theme(legend.position = "bottom")

ggarrange(p_post, p_rope_pct,
          labels = "AUTO", ncol = 2,
          common.legend = TRUE, legend = "bottom")
```

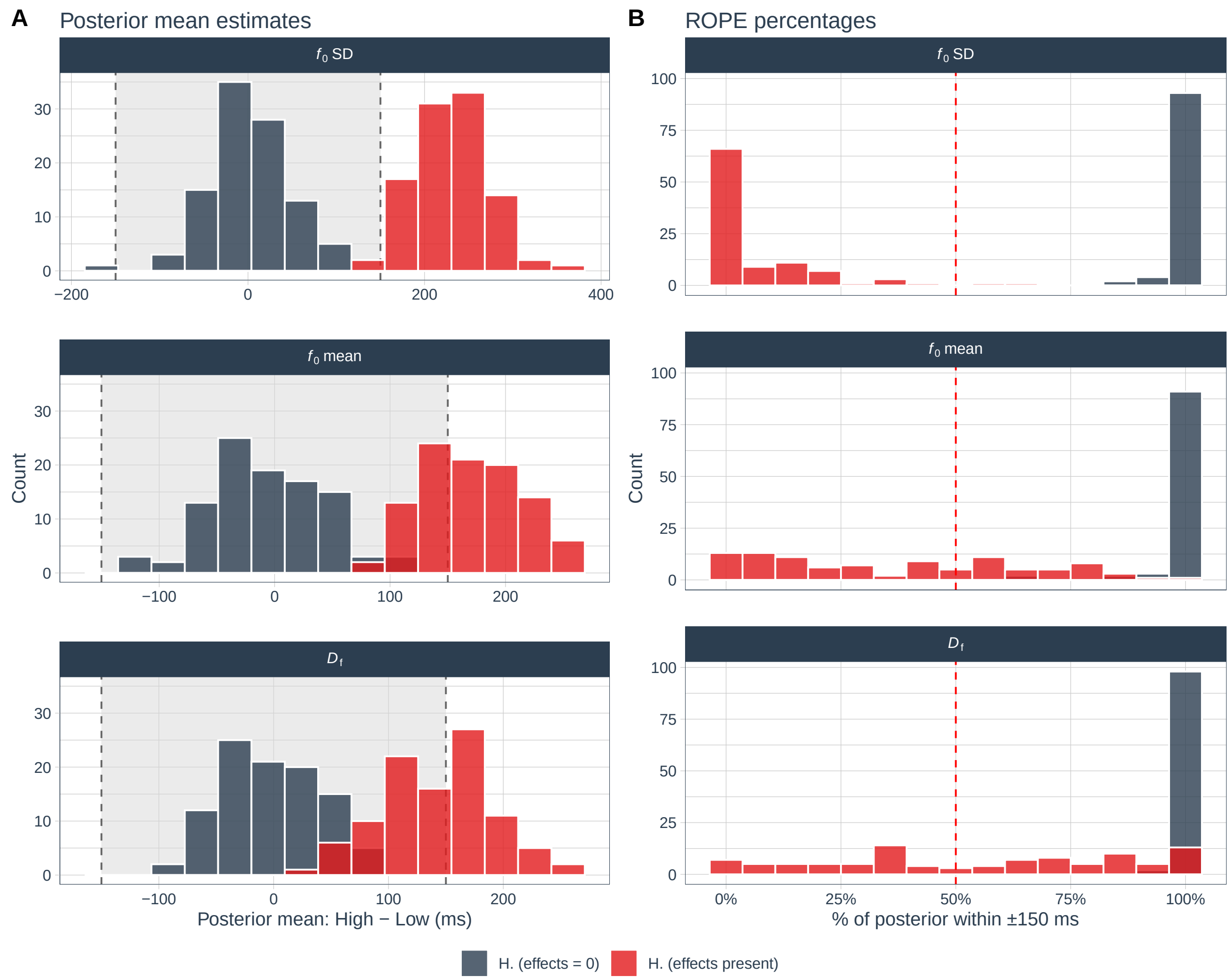

Figure 4. Bayesian posterior estimates and ROPE percentages across 100 simulations at n = 200, under both populations. **Left column:** Posterior mean estimates (High − Low, ms). The grey shaded region marks the ROPE (±150 ms). Under H_1, posteriors should lie well outside the ROPE; under H_0, they should cluster near zero. **Right column:** ROPE percentages. The dashed line marks 50%. High ROPE % under H_0 and low ROPE % under H_1 confirms that the model discriminates correctly.

7 Overall Summary

```r
detect_at_target |>
  left_join(equiv_at_target, by = c("term", "n_sample")) |>
  mutate(
    Effect = case_when(
      term == "f0_sd"   ~ "$f_0$ SD (H1)",
      term == "f0_mean" ~ "$f_0$ mean (H2)",
      term == "Df"      ~ "$D_f$ (H3)"
    ),
    `Expected effect` = case_when(
      term == "f0_sd"   ~ paste0(eff_f0sd,   " ms"),
      term == "f0_mean" ~ paste0(eff_f0mean, " ms"),
      term == "Df"      ~ paste0(eff_Df,     " ms")
    ),
    `Detection power`  = paste0(round(power_detect * 100, 1), "\\%"),
    `Equiv. bounds`    = paste0("\u00b1", eq_bound, " ms"),
    `Equiv. power`     = paste0(round(power_equiv  * 100, 1), "\\%")
  ) |>
  select(Effect, `Expected effect`, `Detection power`,
         `Equiv. bounds`, `Equiv. power`) |>
  tt(notes = paste0(
      "\\textit{Note.} Detection power: frequentist LMM, $\\alpha = .05$,
      from H$_1$ simulations. ",
      "Equivalence power: 90\\% CI within $\\pm ", eq_bound,
      "$ ms, from H$_0$ simulations. ",
      "Bayesian ROPE analysis uses the same bounds; see the Bayesian
      Sensitivity Analysis section.")) |>
  tt_style(align = "lcccc") |>
  format_tt(escape = FALSE)
```

Table 5. Power summary for the study at the pre-specified target sample size ($n = 200$). Detection power is from H_1 Monte Carlo simulations; equivalence power from H_0 simulations. The study is well powered for detection of all three confirmatory effects. Equivalence power is modest but transparent, reflecting the high trial-level noise of infant eye-tracking (see Section 3).

Effect	Expected effect	Detection power	Equiv. bounds	Equiv. power
D_f (H3)	50 ms	83%	±150 ms	83%
f_0 mean (H2)	75 ms	97%	±150 ms	86%
f_0 SD (H1)	100 ms	100%	±150 ms	85%

Note. Detection power: frequentist LMM, $\alpha = .05$, from H_1 simulations. Equivalence power: 90% CI within ±150 ms, from H_0 simulations. Bayesian ROPE analysis uses the same bounds; see the Bayesian Sensitivity Analysis section.

The simulation confirms that:

- At $n = \mathbf{200}$, detection power exceeds 80% for all three confirmatory main effects under the assumed population effect sizes.

- **Equivalence bounds** are set at ± 150 ms, anchored to the precision limit of the design: the smallest effect detectable by a practically feasible infant eye-tracking study of this type, given $\sigma \approx 1000$ ms. This corresponds to 3% of the total fixation-time range and 150% of the largest expected main effect[17,18].
- **Equivalence power** at $n = 200$ is approximately 85% on average, meeting the 80% target.
- The **frequentist equivalence test** generalises TOST to LMM fixed effects via model-based `emmeans` CIs, and detection and equivalence power are estimated from separate superpopulations as required for correct interpretation[14].
- The **Bayesian ROPE analysis** uses `brms` with weakly informative priors and `bayestestR`'s `equivalence_test()`[7], reported as a secondary analysis in the main Registered Report.
- **Interaction effects** are not powered at this sample size and are treated as exploratory throughout.

8 Session Information

```
pander::pander(sessionInfo())
```

R version 4.5.3 (2026-03-11)

Platform: x86_64-pc-linux-gnu

locale: *LC_CTYPE=en_GB.UTF-8*, *LC_NUMERIC=C*, *LC_TIME=es_CO.UTF-8*, *LC_COLLATE=en_GB.UTF-8*, *LC_MONETARY=es_CO.UTF-8*, *LC_MESSAGES=en_GB.UTF-8*, *LC_PAPER=es_CO.UTF-8*, *LC_NAME=C*, *LC_ADDRESS=C*, *LC_TELEPHONE=C*, *LC_MEASUREMENT=es_CO.UTF-8* and *LC_IDENTIFICATION=C*

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *cmdstanr(v.0.9.0)*, *posterior(v.1.6.1)*, *bayestestR(v.0.17.0)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.1)*, *tinytable(v.0.16.0)*, *scales(v.1.4.0)*, *ggpubr(v.0.6.3)*, *PerformanceAnalytics(v.2.0.8)*, *quantmod(v.0.4.28)*, *TTR(v.0.24.4)*, *xts(v.0.14.2)*, *zoo(v.1.8-15)*, *tidyquant(v.1.0.11)*, *ggdist(v.3.3.3)*, *furrr(v.0.3.1)*, *future(v.1.69.0)*, *effectsize(v.1.0.1)*, *emmeans(v.2.0.2)*, *lmerTest(v.3.2-1)*, *lme4(v.2.0-1)*, *Matrix(v.1.7-4)*, *lubridate(v.1.9.5)*, *forcats(v.1.0.1)*, *stringr(v.1.6.0)*, *dplyr(v.1.2.0)*, *purrr(v.1.2.1)*, *readr(v.2.2.0)*, *tidyr(v.1.3.2)*, *tibble(v.3.3.1)*, *ggplot2(v.4.0.2)* and *tidyverse(v.2.0.0)*

loaded via a namespace (and not attached): *Rdpack(v.2.6.6)*, *sandwich(v.3.1-1)*, *rlang(v.1.1.7)*, *magrittr(v.2.0.4)*, *multcomp(v.1.4-29)*, *otel(v.0.2.0)*, *matrixStats(v.1.5.0)*, *compiler(v.4.5.3)*, *loo(v.2.9.0)*, *vctrs(v.0.7.1)*, *quadprog(v.1.5-8)*, *pkgconfig(v.2.0.3)*, *fastmap(v.1.2.0)*, *backports(v.1.5.0)*, *pander(v.0.6.6)*, *rmarkdown(v.2.30)*, *tzdb(v.0.5.0)*, *ps(v.1.9.1)*, *nloptr(v.2.2.1)*, *xfun(v.0.56)*, *jsonlite(v.2.0.0)*, *broom(v.1.0.12)*, *parallel(v.4.5.3)*, *R6(v.2.6.1)*, *stringi(v.1.8.7)*, *RColorBrewer(v.1.1-3)*, *parallelly(v.1.46.1)*, *car(v.3.1-5)*, *boot(v.1.3-32)*, *numDeriv(v.2016.8-1.1)*, *estimability(v.1.5.1)*, *knitr(v.1.51)*, *parameters(v.0.28.3)*, *bayesplot(v.1.15.0)*, *splines(v.4.5.3)*, *timechange(v.0.4.0)*, *tidyselect(v.1.2.1)*, *rstudioapi(v.0.18.0)*, *abind(v.1.4-8)*, *yaml(v.2.3.12)*, *codetools(v.0.2-20)*, *processx(v.3.8.6)*, *curl(v.7.0.0)*, *listenv(v.0.10.0)*, *lattice(v.0.22-9)*, *withr(v.3.0.2)*, *bridgesampling(v.1.2-1)*, *S7(v.0.2.1)*, *coda(v.0.19-4.1)*, *evaluate(v.1.0.5)*, *survival(v.3.8-6)*, *RcppParallel(v.5.1.11-2)*, *pillar(v.1.11.1)*, *tensorA(v.0.36.2.1)*, *carData(v.3.0-6)*, *checkmate(v.2.3.4)*, *reformulas(v.0.4.4)*, *insight(v.1.4.6)*, *distributional(v.0.7.0)*, *generics(v.0.1.4)*, *hms(v.1.1.4)*, *rstantools(v.2.6.0)*, *minqa(v.1.2.8)*, *globals(v.0.19.0)*, *xtable(v.1.8-8)*, *glue(v.1.8.0)*, *tools(v.4.5.3)*, *ggsignif(v.0.6.4)*, *mvtnorm(v.1.3-3)*, *grid(v.4.5.3)*, *rbibutils(v.2.4.1)*, *RobStatTM(v.1.0.11)*, *datawizard(v.1.3.0)*, *nlme(v.3.1-168)*, *Formula(v.1.2-5)*, *cli(v.3.6.5)*, *Brobdingnag(v.1.2-9)*, *gtable(v.0.3.6)*, *rstatix(v.0.7.3)*, *digest(v.0.6.39)*, *TH.data(v.1.1-5)*, *farver(v.2.1.2)*, *htmltools(v.0.5.9)*, *lifecycle(v.1.0.5)* and *MASS(v.7.3-65)*

References

1. Wickham, H. *et al.* Welcome to the tidyverse. *Journal of Open Source Software* **4,** 1686 (2019).
2. Kuznetsova, A., Brockhoff, P. B. & Christensen, R. H. B. lmerTest Package: Tests in Linear Mixed Effects Models. *Journal of Statistical Software* **82,** 1–26 (2017).
3. Lenth, R. V. & Piaskowski, J. *emmeans: Estimated Marginal Means, aka Least-Squares Means* R package version 2.0.2 (2026). https://CRAN.R-project.org/package=emmeans.

4. Ben-Shachar, M. S., Lüdecke, D. & Makowski, D. effectsize: Estimation of Effect Size Indices and Standardized Parameters. *Journal of Open Source Software* **5,** 2815. https://doi.org/10.21105/joss.02815 (2020).
5. Vaughan, D. & Dancho, M. *furrr: Apply Mapping Functions in Parallel using Futures* R package version 0.3.1 (2022). https://CRAN.R-project.org/package=furrr.
6. Bürkner, P.-C. Advanced Bayesian Multilevel Modeling with the R Package brms. *The R Journal* **10,** 395–411 (2018).
7. Makowski, D., Ben-Shachar, M. S. & Lüdecke, D. bayestestR: Describing Effects and their Uncertainty, Existence and Significance within the Bayesian Framework. *Journal of Open Source Software* **4,** 1541. https://joss.theoj.org/papers/10.21105/joss.01541 (2019).
8. Kay, M. ggdist: Visualizations of Distributions and Uncertainty in the Grammar of Graphics. *IEEE Transactions on Visualization and Computer Graphics* **30,** 414–424 (2024).
9. Kay, M. *ggdist: Visualizations of Distributions and Uncertainty* R package version 3.3.3 (2025). https://mjskay.github.io/ggdist/.
10. Dancho, M. & Vaughan, D. *tidyquant: Tidy Quantitative Financial Analysis* R package version 1.0.11 (2025). https://CRAN.R-project.org/package=tidyquant.
11. Kassambara, A. *ggpubr: 'ggplot2' Based Publication Ready Plots* R package version 0.6.3 (2026). https://CRAN.R-project.org/package=ggpubr.
12. Arel-Bundock, V. *tinytable: Simple and Configurable Tables in 'HTML', 'LaTeX', 'Markdown', 'Word', 'PNG', 'PDF', and 'Typst' Formats* R package version 0.16.0 (2026). https://CRAN.R-project.org/package=tinytable.
13. Gabry, J., Češnovar, R., Johnson, A. & Bronder, S. *cmdstanr: R Interface to 'CmdStan'* R package version 0.9.0 (2025). https://mc-stan.org/cmdstanr/.
14. Lakens, D. Equivalence Tests: A Practical Primer for *t* Tests, Correlations, and Meta-Analyses. *Social Psychological and Personality Science* **8,** 355–362. ISSN: 1948-5506, 1948-5514. (2025) (May 2017).
15. John K. Kruschke. *Doing Bayesian Data Analysis: A Tutorial with R, JAGS, and Stan* 2nd ed. (Academic Press, Amsterdam, Nov. 2015).
16. Field, A. *Discovering Statistics Using IBM SPSS Statistics: And Sex and Drugs and Rock 'n' Roll* 4th edition. ISBN: 9781446274583 (Sage, Los Angeles London New Delhi Singapore Washington DC, 2013).
17. Simonsohn, U. Small Telescopes: Detectability and the Evaluation of Replication Results. *Psychological Science* **26,** 559–569. ISSN: 0956-7976. (2024) (May 2015).
18. Lakens, D., Scheel, A. M. & Isager, P. M. Equivalence Testing for Psychological Research: A Tutorial. *Advances in Methods and Practices in Psychological Science* **1,** 259–269. ISSN: 2515-2459, 2515-2467. (2020) (June 2018).